

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

GUÍA DE PRÁCTICA EXPERIMENTAL

1 DATOS GENERALES

CARRERA: SOFTWARE (REDISEÑO)

ASIGNATURA: APLICACIONES WEB [111]

NIVEL / PARALELO: 5TO NIVEL – A

N.º DE ESTUDIANTES:

PROFESOR: GUERRERO ULLOA GLEISTON CICERON

PERIODO ACADÉMICO: 2026-2027 PPA

FECHA: 01-05-2026

TIPO DE GUÍA: PRÁCTICA EXPERIMENTAL

TIPO DE UBICACIÓN: INTERNA

TIPO AULA/LABORATORIO: LABORATORIOS

AULA/LABORATORIO: FCCDD-LAB-TIC-201

UBICACIÓN: CAMPUS CENTRAL

TIPO DE APRENDIZAJE: Aplicación de contenidos conceptuales

N. HORAS: 12

2 DATOS ACADÉMICOS

Resultado de aprendizaje

Construye una aplicación web completa bajo el patrón MVC utilizando un framework moderno, exponiendo e integrando servicios web REST y SOAP, y aplicando criterios de calidad, seguridad e interoperabilidad en un proyecto integrador contextualizado en necesidades reales del entorno productivo.

Tema de la práctica

Modelo Vista Controlador y Servicios Web: Aplicación MVC Completa con API REST, Autenticación JWT y Consumo de APIs Externas

3 OBJETIVO

Objetivo General

Construir la versión completa del Proyecto Fin de Curso como aplicación web MVC con framework moderno, exponiendo una API REST propia documentada con OpenAPI/Swagger (mínimo 5 recursos con CRUD completo), autenticación JWT stateless, consumo de al menos una API externa con gestión de errores y caché, y preparar la defensa oral que demuestre el dominio técnico de todas las competencias de la asignatura.

Objetivos Específicos:

- OE1. **Implementar** la aplicación MVC completa del PFC con el framework elegido, incluyendo mínimo 4 entidades con CRUD completo, autenticación con roles y permisos, y mínimo 10 pruebas de feature o integration test.
- OE2. **Diseñar e implementar** una API REST propia siguiendo los 6 principios de Fiel ding, con autenticación JWT, respuestas JSON estructuradas ({success, data, message, errors, meta}), documentación Swagger UI accesible, y colección Postman exportada.
- OE3. **Integrar** al menos una API REST externa relevante para el PFC con cliente HTTP del servidor, caché de respuestas en Redis, manejo robusto de errores (timeout, 4xx, 5xx) y mostrar los datos consumidos en la interfaz.
- OE4. **Presentar** oralmente el PFC en la Semana 17 demostrando en vivo los flujos principales, la API en Swagger UI, las métricas de rendimiento y seguridad obtenidas, y los fundamentos técnicos de las decisiones tomadas.

4 FUNDAMENTO TEÓRICO

Importante

El informe de PE-U4 es el informe técnico final del PFC. Debe desarrollar el fundamento teórico completo de la Unidad IV con síntesis crítica propia, e integrar referencias a los fundamentos de las Unidades I-III cuando sea relevante. Este informe debe ser de calidad suficiente para ser presentado en un congreso universitario.

5.1 El Patrón MVC y su Implementación en Frameworks Modernos

El equipo debe desarrollar (mínimo 350 palabras):

Historia y evolución del MVC: describir el MVC original de Reenskaug (1979) en Smalltalk-80: modelo como estado, vista como representación, controlador como manejador de entrada. Explicar cómo el modelo web adaptó el MVC: el controlador maneja solicitudes HTTP (no eventos de teclado/ratón), la vista se renderiza en el servidor (template engine) o en el cliente (SPA), el modelo incluye repositorios y servicios. Describir las variantes: MVP (Model-View-Presenter, sin referencia directa vista-modelo), MVVM (Model-View-ViewModel, data binding bidireccional para SPA).

El Front Controller pattern: explicar por qué todos los frameworks MVC web usan un punto de entrada único (index.php, Program.cs): centraliza el manejo de errores, la autenticación, el logging y el routing. Describir el ciclo completo de una petición MVC en el framework del PFC (6 pasos con código real).

Comparativa de frameworks MVC en 2025: construir tabla comparativa de Laravel 11.x, Symfony 7.x, Spring Boot 3.x, ASP.NET Core 8.x y Django 5.x con criterios: paradigma (convención sobre configuración vs. explícito), ORM incluido, motor de plantillas, ecosistema de paquetes, rendimiento benchmarked (TechEmpower Framework Benchmarks Round 22), adopción en el mercado latinoamericano.

Justificación del framework del PFC: basándose en los criterios de la tabla, justificar documentadamente la elección del framework con referencia al ADR-001 escrito en PE-U3.

Referencias mínimas IEEE:

T. Otwell, "Laravel 11.x Documentation," Laravel LLC, 2024. [Online]. Available: <https://laravel.com/docs/11.x>
M.Fowler, Patterns of Enterprise Application Architecture. Addison-Wesley, 2002, ISBN: 978-0-321-12521-7.

5.2 Diseño de APIs REST: Principios de Fielding y OpenAPI

El equipo debe desarrollar (mínimo 350 palabras):

Los 6 principios de REST según Fielding (2000): para cada principio explicar su significado técnico y cómo se aplica (o no) en la API del PFC:

1. Client-Server: separación de responsabilidades cliente-servidor
2. Stateless: cada petición contiene toda la información necesaria (JWT)
3. Cacheable: las respuestas pueden ser cacheadas (cabecera Cache-Control)
4. Uniform Interface: URLs que identifican recursos, verbos HTTP semánticos
5. Layered System: el cliente no sabe si habla con un servidor, proxy o CDN
6. Code-On-Demand (opcional): el servidor puede enviar código ejecutable

Diseño de URIs REST semánticas: reglas y anti-patrones: URIs deben ser sustantivos (no verbos): GET /api/v1/usuarios (no /api/v1/getUsuarios). Jerarquía de recursos: GET /api/v1/usuarios/{id}/pedidos. Versionado: por URL (/v1/), por cabecera (Accept-Version), por parámetro de query. Mostrar el diseño completo de las URIs del PFC.

JWT: estructura y flujo de autenticación: describir los tres componentes del JWT: Header (algoritmo HS256/RS256), Payload (claims: sub, iat, exp, roles), Signature. Flujo completo: login → servidor genera JWT firmado → cliente almacena (localStorage o cookie HttpOnly) → cliente envía en cada petición (Authorization: Bearer {token}) → servidor verifica la firma y extrae claims. Analizar el problema de revocación de JWT y las estrategias (token blacklist con Redis, refresh token con rotación).

OpenAPI/Swagger 3.0: describir la especificación OpenAPI como contrato entre el equipo de frontend y backend. Mostrar cómo se documenta un endpoint con YAML/JSON (operationId, parameters, requestBody, responses con schemas y ejemplos). Comparar Swagger UI (interfaz interactiva) con Redoc (documentación estática).

Referencias mínimas IEEE:

R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. [Online]. Available: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
L. Richardson and S. Ruby, RESTful Web Services. Sebastopol, CA, USA: O'Reilly Media, 2007, ISBN: 978-0-596-52926-0.

5.3 Consumo de Servicios Web Externos y SOAP

El equipo debe desarrollar (mínimo 350 palabras):

SOAP vs. REST: análisis técnico comparativo: describir la arquitectura SOAP: sobre XML, protocolo WSDL para descripción, transporte sobre HTTP (o SMTP/TCP), acción SOAP. Comparar con REST: sobre cualquier formato (JSON preferido), descripción opcional (OpenAPI), solo HTTP. Tabla comparativa: formato del mensaje, overhead de serialización, tipado, interoperabilidad, facilidad de uso desde JavaScript, casos de uso vigentes de SOAP en 2025 (banca, sector público ecuatoriano, servicios del SRI).

Cientes HTTP del lado del servidor: comparar GuzzleHTTP (PHP), HttpClient (.NET), RestTemplate y WebClient (Java/Spring Boot): manejo de timeout, reintentos (retry policy con backoff exponencial), concurrencia. Mostrar fragmento de código del consumo de la API externa del PFC con manejo completo de errores.

Caché de respuestas externas: justificar por qué las APIs externas deben cachearse: evitar superar límites de rate limiting, reducir latencia, proteger contra indisponibilidad del servicio externo. Mostrar la implementación del patrón cache aside para la API externa del PFC con el TTL apropiado al dominio (datos meteorológicos: 10 min; datos de divisas: 1 hora; datos estáticos: 24 horas).

Tendencias: Jamstack y APIs en 2025: describir el patrón Jamstack (JavaScript + APIs + Markup): generadores de sitios estáticos (Next.js, Nuxt.js, Astro) que consumen APIs en build time o runtime. Analizar cómo la IA generativa está cambiando el desarrollo web: GitHub Copilot, herramientas de scaffolding inteligente, riesgos éticos (propiedad intelectual del código generado, responsabilidad del desarrollador).

Referencias mínimas IEEE:

R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, UC Irvine, 2000. [Online]. Available: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

H. Washizaki, Ed., SWEBOOK Guide v4.0. IEEE Computer Society, 2024.

5.4 Seguridad, Pruebas de Rendimiento y Despliegue con Docker

El equipo debe desarrollar (mínimo 350 palabras):

Auditoría de seguridad OWASP en la aplicación MVC completa: para cada vulnerabilidad del OWASP Top 10 A01–A07 y XSS: describir cómo se manifestaría en la aplicación MVC del PFC y qué implementaron para mitigarla. Incluir tabla resumen con: vulnerabilidad, vector de ataque, contramedida implementada, evidencia de la implementación (fragmento de código o configuración).

Prueba de carga básica con Apache Bench: describir la metodología de prueba de carga: número de usuarios concurrentes (c), total de solicitudes (n), endpoint representativo. Las métricas P50, P95 y P99 del tiempo de respuesta, el throughput en req/s y la tasa de error. Mostrar el comando usado y el resultado obtenido.

Containerización con Docker Compose: describir los beneficios de Docker para el despliegue de aplicaciones web: entornos reproducibles, aislamiento de dependencias, facilidad de CI/CD. Para el PFC: explicar el Docker Compose con los servicios (app, mysql, redis, nginx) y sus relaciones. Describir las variables de entorno necesarias y por qué nunca deben estar en el código fuente.

Reflexión sobre el proceso de aprendizaje: cada integrante debe escribir un párrafo (mínimo 100 palabras) sobre las lecciones técnicas más importantes aprendidas en el desarrollo del PFC, los errores más costosos que cometieron y cómo los resolverían de diferente manera en un proyecto futuro.

Referencias mínimas IEEE:

OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: <https://owasp.org/Top10/>

M. T. Nygard, Release It!, 2nd ed. Pragmatic Bookshelf, 2018, ISBN: 978-1-68050 239-8.

5 MATERIALES, EQUIPOS, INSUMOS Y REACTIVOS

Materiales

- Framework MVC elegido: Laravel 11.x / ASP.NET Core 8.x / Spring Boot 3.x
- Docker Desktop 4.x con Docker Compose 2.x
- Apache Bench: incluido en Apache HTTPD o XAMPP
- Postman (app de escritorio): <https://www.postman.com/downloads/>
- Navegador Google Chrome (para la defensa en vivo)

Equipos

- Proyector/datashow y control, automático.
- Cable HDMI/adaptador para proyección durante la defensa
- Intel Core i5 o AMD Ryzen 5 (mínimo), i7/Ryzen 7 RAM 16 GB HD 20GB.

Insumos y reactivos

- JSONPlaceholder(pruebas de registro): <https://jsonplaceholder.typicode.com>
- La API elegida en el Paso 3 (ver tabla de opciones)
- TechEmpower Benchmarks: <https://www.techempower.com/benchmarks/>
- SecurityHeaders: <https://securityheaders.com>
- JWT.io: <https://jwt.io> (decodificar y verificar tokens JWT)
- Swagger Editor: <https://editor.swagger.io> (para verificar YAML)

6 PROCEDIMIENTOS

Paso 1– Aplicación MVC completa con framework elegido

1.1 Verificación de la arquitectura MVC completa

En este punto, la aplicación MVC debe estar construida sobre el trabajo de las prácticas anteriores. Verificar que la estructura es correcta según lo mostrado en el documento adjunto en la tarea.

1.2 Sistema de roles y permisos (Laravel Breeze + Spatie)

1.3 Pruebas de feature (mínimo 10)

Criterio de verificación: php artisan test reporta ≥ 10 pruebas pasando con 0 fallos. Captura del resultado en el informe.

Paso 2– API REST propia con OpenAPI/Swagger

2.1 Implementar la API REST en Laravel (API Resources)

2.2 Autenticación JWT con Laravel Sanctum

2.3 Documentar con L5-Swagger (Laravel OpenAPI)

Criterio de verificación: Swagger UI está accesible en /api/documentation. To dos los endpoints están documentados con schemas de request/response. La colección Postman está exportada y tiene ≥ 20 requests en carpetas organizadas.

Paso 3– Consumo de API externa con gestión de errores y caché

3.1 Consumo de API externa con GuzzleHTTP

3.2 APIs externas disponibles para el PFC

Elegir/construir una API apropiada para el dominio del PFC.

Criterio de verificación: La API externa se muestra en la interfaz del PFC. El Redis almacena la respuesta cacheada (verificar con redis-cli KEYS "*api_externa*"). Los errores de red se manejan elegantemente (mensaje amigable al usuario).

Paso 4– Seguridad, pruebas de carga y Docker Compose

4.1 Prueba de seguridad: verificar headers HTTP

4.2 Prueba de carga básica con Apache Bench

4.3 Docker Compose de producción

Criterio de verificación: docker compose up-d inicia todos los servicios sin errores. docker compose ps muestra todos en estado Up. La aplicación es accesible en http://localhost.

Paso 5– Preparación de la defensa oral (Semana 17)

La defensa oral es la Evaluación Parcial del Segundo Corte (Semana 17). Todos los integrantes del equipo deben poder responder preguntas técnicas sobre cualquier parte del sistema.

5.1 Estructura de la presentación (15 minutos)

1. [2 min] Descripción del sistema: ¿Qué problema resuelve el PFC? Diagrama C4 Nivel 1 (contexto). Tecnologías principales.
2. [3 min] Arquitectura: Diagrama C4 Nivel 2 (contenedores). Los 3 ADR más importantes. Decisiones técnicas clave.
3. [4 min] Demostración en vivo: Flujo de autenticación. Operación CRUD principal. Consumo de API externa visible en la UI.
4. [3 min] API REST: Abrir Swagger UI y ejecutar 2–3 endpoints en vivo. Mostrar la colección Postman organizada.
5. [2 min] Métricas: Resultados de Lighthouse (PE-U1), cobertura de pruebas (PE-U3), resultados de Apache Bench. Tabla comparativa de métricas de rendimiento con y sin caché.
6. [1 min] Conclusión: Lecciones aprendidas más importantes. Trabajo futuro propuesto.

5.2 Preguntas frecuentes que el docente puede hacer Preparar respuestas técnicas para:

- ¿Por qué eligieron ese framework y no otro? (referirse al ADR-001)
- ¿Cómo funciona el JWT de su sistema? Explicar el proceso de login hasta el acceso a una ruta protegida.
- ¿Qué pasaría si Redis falla?
- ¿Cómo lo detectaría y cómo respondería la aplicación? Mostrar en el código cómo previenen inyección SQL.
- ¿Cómo escalaría el sistema a 10.000 usuarios concurrentes?

Criterio de verificación: La presentación dura entre 13 y 17 minutos. Todos los integrantes exponen al menos 4 minutos. El sistema funciona en vivo durante la demostración.

7

BIBLIOGRAFÍA

- T.Otwell, "Laravel11.xDocumentation,"LaravelLLC,2024. [Online].Available: <https://laravel.com/docs/11.x>
- OWASPFoundation, "OWASPTop10:2021," 2021. [Online].Available: <https://owasp.org/Top10/>
- H.Washizaki,Ed.,GuidetotheSoftwareEngineeringBodyofKnowledge(SWE BOK)Version4.0.Piscataway,NJ,USA: IEEEComputerSociety,2024.
- M.T.Nygard,ReleaseIt!:DesignandDeployProduction-ReadySoftware, 2nd ed.Raleigh,NC,USA:PragmaticBookshelf,2018, ISBN: 978-1-68050-239-8.

8 ANEXOS

- AnexoA—Estructura del repositorio (versión final en main) Como se muestra en el archivo adjunto en la tarea.
- AnexoC—Preguntas de Análisis
- [Evaluación/Bloom nivel 6]: Revisando el PFC completo (PE-U1aPE-U4): ¿cuál fue la decisión técnica más difícil que tomó el equipo? ¿Qué alternativas consideraron? ¿Con el conocimiento actual, tomarían la misma decisión o una diferente? Justifique con criterios técnicos referenciados.
- [Síntesis / Bloom nivel 5]: Los resultados de Apache Bench mostraron ___ req/s con ___% de error. Si se exigiera que el sistema soporte 500 usuarios concurrentes con tasa de error 0%, ¿qué cambios de arquitectura y código serían necesarios? Dibuje la arquitectura escalada.
- [Análisis / Bloom nivel 4]: Analice los 6 principios REST de Fielding contra la API REST del PFC: ¿cuáles cumple completamente, cuáles parcialmente y cuáles no? Para los principios no cumplidos: ¿cuál es el costo de no cumplirlos en este contexto específico?
- [Evaluación / Bloom nivel 6]: Comparando el PFC con un sistema web de producción real del mismo dominio (buscar un caso de estudio publicado), ¿qué características de calidad (ISO/IEC 25010:2023) están cubiertas por el PFC y cuáles requerirían trabajo adicional para ser comercialmente viables? ¿Cuánto tiempo estimaría para hacerlo comercialmente viable?

9 RESULTADOS DE ESTUDIANTES

EQUIPO A

ZAMBRANO MOREIRA ALISON ARIANA
UMAGINGA AREVALO JEFFERSON MANUEL
MONCAYO LOOR XAVIER ALEJANDRO

Resultados obtenidos

Tercer Avance del Proyecto — App Web

Sistema de gestión bibliotecaria que automatiza los procesos de autenticación, catálogo, préstamos, reservas, multas y reportes. Aplicación web con arquitectura cliente-servidor.

Stack Tecnológico

- **Backend:** Java 21 + Spring Boot 3.4 (API REST, JWT en cookie HttpOnly)
- **Frontend:** Laravel 13 (Bootstrap 5, Blade)
- **Base de datos:** PostgreSQL 16 + Redis 7 (caché)
- **Infraestructura:** Docker Compose (PostgreSQL, Redis, API, Frontend, pgAdmin)

Avances del Proyecto

- **Primer avance (v0.3.0):** SRS, casos de uso, arquitectura C4
- **Segundo avance (v0.7.0):** Autenticación JWT, CRUD de usuarios y libros, préstamos y devoluciones, Docker Compose
- **Tercer avance (v0.9.0-rc):** Claims JWT estándar, errores RFC 7807 (ProblemDetail), caché Redis, stored procedures, ingeniería de requisitos (ISO/IEC/IEEE 29148), historias de usuario, matriz de trazabilidad, pruebas automatizadas (JUnit 5, JaCoCo), CI/CD (GitHub Actions), pruebas de carga (k6), reproducibilidad (Makefile, CITATION.cff, digests de imágenes)

Estructura del Repositorio

```
codigo-fuente/  
├── proyecto-pfc/  ← Índice del proyecto, diagramas C4, colección Postman  
├── frontend/      ← Frontend Laravel (vistas Blade que consumen la API)  
├── api/           ← Backend Spring Boot (controllers, services, security, entities, tests)  
├── database/      ← Schema, seed y stored procedures PostgreSQL  
└── endpoints/    ← Documento con todos los endpoints de la API
```

Ruta

codigo-fuente/proyecto-pfc/
codigo-fuente/frontend/
codigo-fuente/api/
codigo-fuente/database/
codigo-fuente/endpoints/
docs/
scripts/
docker-compose.yml

Contenido

Índice del proyecto, diagramas C4, colección Postman
Frontend Laravel
Backend Spring Boot
Schema, seed y stored procedures
Lista de endpoints de la API
SRS, casos de uso, historias, matriz, observaciones
Pruebas de carga (k6), análisis de rendimiento, validación CI
Infraestructura completa

Conclusión

Tercer Avance del Proyecto — App Web

Sistema de gestión bibliotecaria que automatiza los procesos de autenticación, catálogo, préstamos, reservas, multas y reportes. Aplicación web con arquitectura cliente-servidor.

Stack Tecnológico

- **Backend:** Java 21 + Spring Boot 3.4 (API REST, JWT en cookie HttpOnly)
- **Frontend:** Laravel 13 (Bootstrap 5, Blade)
- **Base de datos:** PostgreSQL 16 + Redis 7 (caché)
- **Infraestructura:** Docker Compose (PostgreSQL, Redis, API, Frontend, pgAdmin)

Avances del Proyecto

- **Primer avance (v0.3.0):** SRS, casos de uso, arquitectura C4
- **Segundo avance (v0.7.0):** Autenticación JWT, CRUD de usuarios y libros, préstamos y devoluciones, Docker Compose
- **Tercer avance (v0.9.0-rc):** Claims JWT estándar, errores RFC 7807 (ProblemDetail), caché Redis, stored procedures, ingeniería de requisitos (ISO/IEC/IEEE 29148), historias de usuario, matriz de trazabilidad, pruebas automatizadas (JUnit 5, JaCoCo), CI/CD (GitHub Actions), pruebas de carga (k6), reproducibilidad (Makefile, CITATION.cff, digests de imágenes)

Estructura del Repositorio

```
codigo-fuente/  
├── proyecto-pfc/  ↳ Índice del proyecto, diagramas C4, colección Postman  
├── frontend/      ↳ Frontend Laravel (vistas Blade que consumen la API)  
├── api/           ↳ Backend Spring Boot (controllers, services, security, entities, tests)  
├── database/      ↳ Schema, seed y stored procedures PostgreSQL  
└── endpoints/    ↳ Documento con todos los endpoints de la API
```

Ruta

codigo-fuente/proyecto-pfc/
codigo-fuente/frontend/
codigo-fuente/api/
codigo-fuente/database/
codigo-fuente/endpoints/
docs/
scripts/
docker-compose.yml

Contenido

Índice del proyecto, diagramas C4, colección Postman
Frontend Laravel
Backend Spring Boot
Schema, seed y stored procedures
Lista de endpoints de la API
SRS, casos de uso, historias, matriz, observaciones
Pruebas de carga (k6), análisis de rendimiento, validación CI
Infraestructura completa

EQUIPO B

TAIPE MORA ZAIDA MELISSA
PANAMA MURILLO MOISES ANTONIO
CAJAS IBARRA IRVIN MARCELO

aa
aa
aaaaaaaaaaaaaaaa

Conclusión

aa
aa

EQUIPO F

CRUZ PEREZ JUSTYN KEITH
BELTRAN MONTIEL FRED ADRIAN
PALLO PINTO DANIEL ALEJANDRO

Resultados obtenidos

Objetivo General

Construir la versión completa del Proyecto Fin de Curso como aplicación web MVC con framework moderno, exponiendo una API REST propia documentada con OpenAPI/Swagger (mínimo 5 recursos con CRUD completo), autenticación JWT stateless, consumo de al menos una API externa con gestión de errores y caché, y preparar la defensa oral que demuestre el dominio técnico de todas las competencias de la asignatura.

Objetivos Específicos:

- OE1. **Implementar** la aplicación MVC completa del PFC con el framework elegido, incluyendo mínimo 4 entidades con CRUD completo, autenticación con roles y permisos, y mínimo 10 pruebas de feature o integration test.
- OE2. **Diseñar e implementar** una API REST propia siguiendo los 6 principios de Fierding, con autenticación JWT, respuestas JSON estructuradas ({success, data, message, errors, meta}), documentación Swagger UI accesible, y colección Postman exportada.
- OE3. **Integrar** al menos una API REST externa relevante para el PFC con cliente HTTP del servidor, caché de respuestas en Redis, manejo robusto de errores (timeout, 4xx, 5xx) y mostrar los datos consumidos en la interfaz.
- OE4. **Presentar** oralmente el PFC en la Semana 17 demostrando en vivo los flujos principales, la API en Swagger UI, las métricas de rendimiento y seguridad obtenidas, y los fundamentos técnicos de las decisiones tomadas.

Conclusión

Objetivo General

Construir la versión completa del Proyecto Fin de Curso como aplicación web MVC con framework moderno, exponiendo una API REST propia documentada con OpenAPI/Swagger (mínimo 5 recursos con CRUD completo), autenticación JWT stateless, consumo de al menos una API externa con gestión de errores y caché, y preparar la defensa oral que demuestre el dominio técnico de todas las competencias de la asignatura.

Objetivos Específicos:

- OE1. **Implementar** la aplicación MVC completa del PFC con el framework elegido, incluyendo mínimo 4 entidades con CRUD completo, autenticación con roles y permisos, y mínimo 10 pruebas de feature o integration test.
- OE2. **Diseñar e implementar** una API REST propia siguiendo los 6 principios de Fierding, con autenticación JWT, respuestas JSON estructuradas ({success, data, message, errors, meta}), documentación Swagger UI accesible, y colección Postman exportada.
- OE3. **Integrar** al menos una API REST externa relevante para el PFC con cliente HTTP del servidor, caché de respuestas en Redis, manejo robusto de errores (timeout, 4xx, 5xx) y mostrar los datos consumidos en la interfaz.
- OE4. **Presentar** oralmente el PFC en la Semana 17 demostrando en vivo los flujos principales, la API en Swagger UI, las métricas de rendimiento y seguridad obtenidas, y los fundamentos técnicos de las decisiones tomadas.

EQUIPO G

FIGUEROA MORALES BRYAN JAVIER
VELEZ LOPEZ RICARDO ELIAS
ROMERO MENDEZ BRYAN STEVEN

Resultados obtenidos

GA -- Práctica Experimental Unidad IV (Grup PFC):

El equipo investiga y desarrolla con rigor científico-bibliográfico el contenido completo de la Unidad IV (Temas 13--16).

Directrices:

- (1) Realizar la revisión bibliográfica de al menos 5 fuentes académicas en norma APA (incluir la disertación de Fielding 2000 sobre REST y documentación oficial del framework elegido); las citas deben aparecer en el cuerpo del informe.
- (2) Aplicación MVC completa: presentar el PFC con todos los módulos de la Entrega 1B funcionando; el docente verificará la aplicación en tiempo real durante la sesión de laboratorio.
- (3) API REST documentada: Swagger UI debe estar accesible en /api/docs; ejecutar al menos 3 endpoints distintos en vivo con Postman o Insomnia durante la sesión.
- (4) Comparativa SOAP vs. REST: elaborar una tabla de al menos 8 criterios (formato del mensaje, WSDL, overhead, seguridad, facilidad de consumo desde móvil, casos de uso actuales en Ecuador); incluir un ejemplo de llamada SOAP y su equivalente REST.
- (5) Reflexión sobre tendencias: escribir una sección de al menos 400 palabras sobre cómo Jamstack, PWA e IA generativa están cambiando el desarrollo web, con ejemplos concretos y posición crítica propia.
- (6) Redactar el informe técnico final con portada, resumen bilingüe (español e inglés), introducción, desarrollo, conclusiones, trabajo futuro y referencias APA. Adjuntar informe en PDF en la Tarea del LMS (SGA) antes del cierre.

Conclusión

GA -- Práctica Experimental Unidad IV (Grup PFC):

El equipo investiga y desarrolla con rigor científico-bibliográfico el contenido completo de la Unidad IV (Temas 13--16).

Directrices:

- (1) Realizar la revisión bibliográfica de al menos 5 fuentes académicas en norma APA (incluir la disertación de Fielding 2000 sobre REST y documentación oficial del framework elegido); las citas deben aparecer en el cuerpo del informe.
- (2) Aplicación MVC completa: presentar el PFC con todos los módulos de la Entrega 1B funcionando; el docente verificará la aplicación en tiempo real durante la sesión de laboratorio.
- (3) API REST documentada: Swagger UI debe estar accesible en /api/docs; ejecutar al menos 3 endpoints distintos en vivo con Postman o Insomnia durante la sesión.
- (4) Comparativa SOAP vs. REST: elaborar una tabla de al menos 8 criterios (formato del mensaje, WSDL, overhead, seguridad, facilidad de consumo desde móvil, casos de uso actuales en Ecuador); incluir un ejemplo de llamada SOAP y su equivalente REST.
- (5) Reflexión sobre tendencias: escribir una sección de al menos 400 palabras sobre cómo Jamstack, PWA e IA generativa están cambiando el desarrollo web, con ejemplos concretos y posición crítica propia.
- (6) Redactar el informe técnico final con portada, resumen bilingüe (español e inglés), introducción, desarrollo, conclusiones, trabajo futuro y referencias APA. Adjuntar informe en PDF en la Tarea del LMS (SGA) antes del cierre.

EQUIPO H

FAJARDO MONTES MICHAEL XAVIER
CARVAJAL LOOR JOHAN STALIN
MARISCAL CABRERA JAIME JOSUE

Resultados obtenidos

Durante el desarrollo de esta práctica se logró avanzar en la integración y mejora del Proyecto Fin de Curso, aplicando los contenidos correspondientes a la Unidad IV sobre el patrón MVC y los servicios web. Se fortaleció la API REST del sistema mediante nuevos recursos y operaciones CRUD, manteniendo la separación entre controladores, servicios, repositorios y entidades. También se ajustó la documentación de la API con Swagger UI, permitiendo su acceso desde /api/docs, y se ampliaron las pruebas automatizadas para comprobar el funcionamiento de los nuevos endpoints, permisos y validaciones. Además, se revisaron aspectos de seguridad relacionados con OWASP, componentes vulnerables y XSS, generando evidencias reales de las verificaciones realizadas. Todo esto permitió obtener una versión más completa, documentada y preparada para su posterior integración y demostración.

Conclusión

La práctica permitió reforzar de manera práctica los conocimientos sobre arquitectura MVC, APIs REST, seguridad y pruebas dentro de una aplicación web real. Trabajar sobre el PFC previamente desarrollado facilitó comprender cómo una aplicación puede evolucionar progresivamente sin necesidad de reconstruir su arquitectura desde cero. Las nuevas implementaciones y validaciones permitieron mejorar la organización del sistema, ampliar sus funcionalidades y mantener controles adecuados de

acceso y manejo de errores. Además, el uso de pruebas automatizadas y herramientas de documentación permitió comprobar que los cambios realizados no afectaran las funcionalidades existentes, dejando una base más sólida para la integración final del proyecto y su presentación.

EQUIPO I

RIOS CUYABAZO JHON KEVIN

ARIAS MOREIRA MAYBELIN GREGORIA

CASTRO PALMA JUSTYN MOISES

Resultados obtenidos

Durante el desarrollo de esta práctica experimental aplicada al proyecto Artisync, se lograron materializar con éxito todos los componentes técnicos requeridos para la Unidad IV. En primer lugar, se consolidó la arquitectura MVC de la aplicación, logrando que todos los módulos correspondientes a la Entrega 1B funcionen de manera fluida y cohesionada. Esto permitió establecer una separación clara entre la lógica de negocio, el manejo de datos y la interfaz de usuario, optimizando el rendimiento general del sistema. Asimismo, se implementó y expuso una API REST robusta, la cual fue documentada exhaustivamente utilizando Swagger UI (accesible mediante la ruta `/api/docs`). A través de esta interfaz, se verificó el correcto funcionamiento de los endpoints mediante pruebas exhaustivas, asegurando que las peticiones y respuestas HTTP cumplan con los estándares de la industria. Paralelamente, la investigación bibliográfica rigurosa y la elaboración de la tabla comparativa entre SOAP y REST proporcionaron un marco teórico sólido que justificó la elección de tecnologías modernas para Artisync. Finalmente, la sección de reflexión crítica sobre Jamstack, PWA e IA generativa permitió alinear el desarrollo del proyecto con las tendencias más vanguardistas del desarrollo web actual.

Conclusión

La culminación de este trabajo práctico demuestra que la aplicación de estándares arquitectónicos modernos es fundamental para la escalabilidad y mantenibilidad de plataformas como Artisync. Se concluye que la adopción del patrón MVC no solo ordenó la base de código de las entregas anteriores, sino que facilitó enormemente la posterior integración de servicios. Por otro lado, la construcción de una API REST documentada con Swagger demostró ser una decisión técnica inmejorable frente a alternativas más rígidas como SOAP; esto se evidenció claramente en la comparativa realizada, donde REST destacó por su menor "overhead", facilidad de consumo desde dispositivos móviles y flexibilidad en el formato de mensajes (JSON). Además, el análisis de las nuevas tendencias tecnológicas subraya que el desarrollo web está en una transición acelerada hacia experiencias más rápidas y ricas (Jamstack y PWA), apoyadas cada vez más por la Inteligencia Artificial. En definitiva, el proyecto Artisync no solo ha cumplido con los requerimientos académicos de esta unidad, sino que ha establecido una base tecnológica robusta, orientada a servicios y preparada para futuras iteraciones e integraciones tecnológicas.

10 RESPONSABLES

GUERRERO ULLOA GLEISTON CICERON
CI: 0913531752
DOCENTE RESPONSABLE

PONCE ORDOÑEZ JESSICA ALEXANDRA
CI: 1205316456
COORDINADOR DE CARRERA